

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра «Програмна інженерія»

Звіт з лабораторної роботи №3

З дисципліни "Генеративні нейронні мережі та їх застосування"

Персоналізація Stable Diffusion за допомогою DreamBooth LoRA та офіційного
training script із diffusers

Виконала:
студентка гр. ПЗМ-25-2
Тимощенко Владислава Русланівна

Перевірила: доцент кафедри ПІ
Шевченко Олена Леонідівна

Харків 2026

1 Мета: персоналізувати Stable Diffusion за допомогою DreamBooth LoRA на невеликому dataset одного об'єкта.

2 Хід роботи

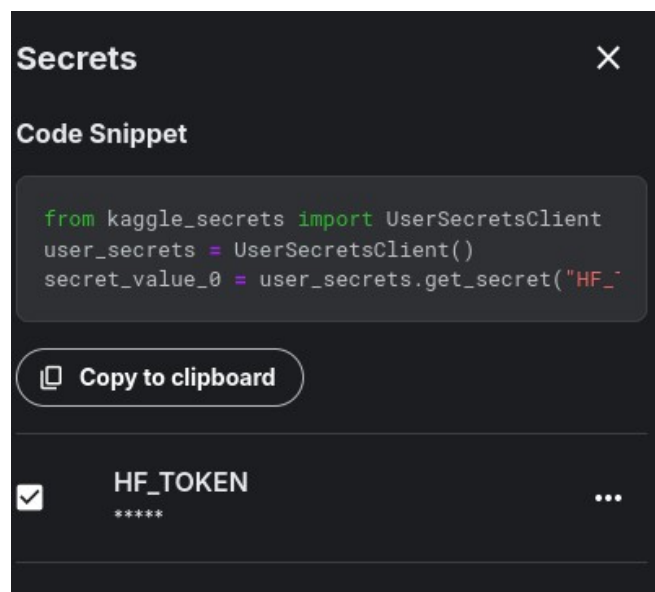
2.1 Відкриємо ноутбук Dreambooth

2.1.1 Авторизація Hugging Face

Авторизуємося в Hugging Face та створимо новий Secret Token з дозволом read

Name	Value	Last Refreshed Date	Last Used Date	Permissions
kaggle-dreambooth	hf_...duOc	less than a minute ago	-	READ

Відкриємо вкладку Secrets та додамо secret з назвою *HF_TOKEN* та увімкнемо доступ ноутбука до цього secret

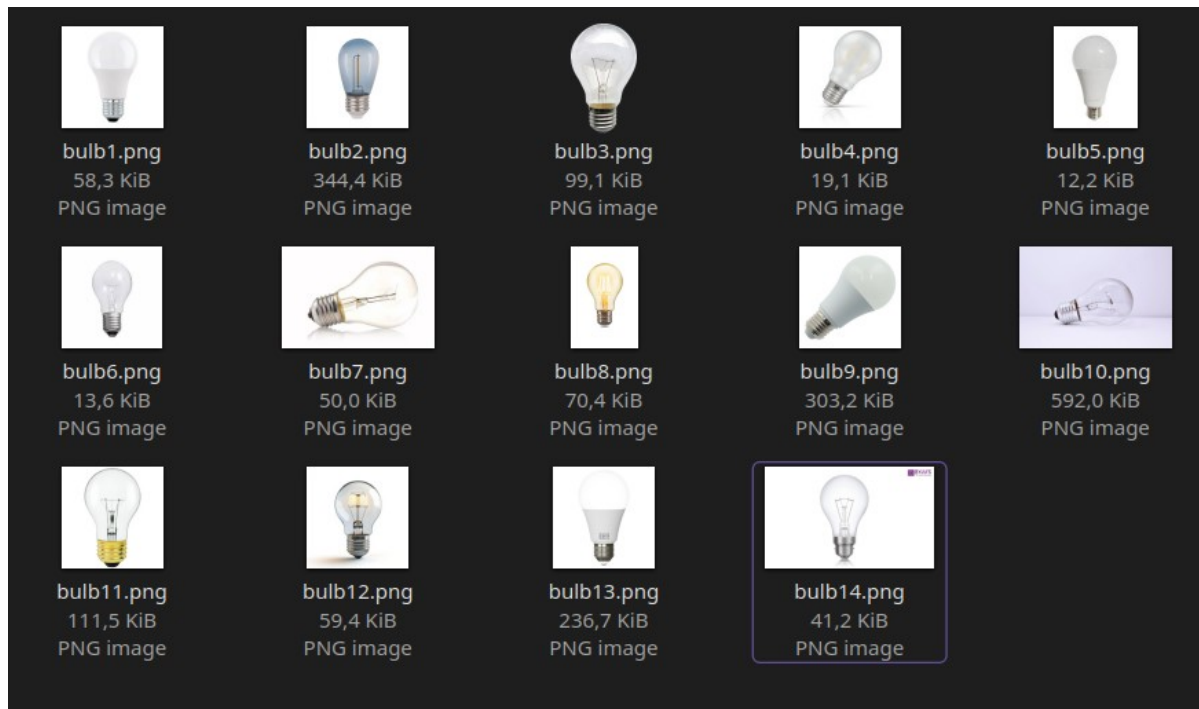


Перевіримо авторизацію

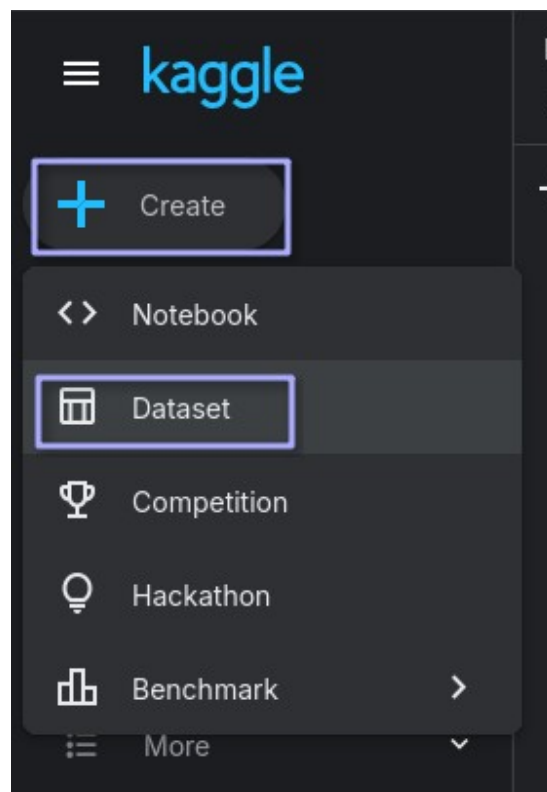
```
Note: Environment variable `HF_TOKEN` is set and is t
HF_TOKEN знайдено в Kaggle Secrets і додано в enviro
Успішна авторизація Hugging Face user: Yowyerwq
```

2.1.2 Підготовка датасету

Підготуємо зображення для створення власного dataset для подальшої роботи. За об'єкт візьмемо лампочку:



Тепер створимо новий датасет в Kaggle



Upload Data

DATASET TITLE
Bulb object
www.kaggle.com/datasets/purplejhvhgcdf/bulb-object

VISIBILITY
Private

FILES

bulb1.png (59.72 kB)	×
bulb2.png (352.69 kB)	×
bulb3.png (101.51 kB)	×
bulb4.png (19.54 kB)	×
bulb5.png (12.53 kB)	×
bulb6.png (13.98 kB)	×
bulb7.png (51.19 kB)	×
bulb8.png (72.07 kB)	×
bulb9.png (310.51 kB)	×
bulb10.png (606.24 kB)	×
bulb11.png (114.15 kB)	×
bulb12.png (60.82 kB)	×
bulb13.png (242.42 kB)	×
bulb14.png (42.14 kB)	×

Reset Create

Змінимо path, щоб той відповідав розташуванню новоствореного датасету

```
# ===== НАЛАШТУВАННЯ ДАТАСЕТУ =====

# True - використати демонстраційний dataset diffusers/dog-example.
# False - використати власний Kaggle dataset із RAW_DATA_DIR.
USE_HF_EXAMPLE_DATASET = False

# Якщо USE_HF_EXAMPLE_DATASET = False, змініть цей шлях на свій Kaggle dataset.
from pathlib import Path

RAW_DATA_DIR = Path("/kaggle/input/datasets/purplejhvhgcdf/bulb-object")

print("RAW_DATA_DIR exists:", RAW_DATA_DIR.exists())
print("Images:", len(list(RAW_DATA_DIR.glob("*"))))
print(list(RAW_DATA_DIR.glob("*"))[:10])

# Папка, куди notebook скопіює/підготує instance images для training script.
INSTANCE_DIR = WORK_DIR / "instance_images"
INSTANCE_DIR.mkdir(parents=True, exist_ok=True)
```

Запустимо цю клітинку для перевірки коректного підключення

```
RAW_DATA_DIR exists: True  
Images: 14
```

Підготуємо датасет:

```
Знайдено 14 файлів. Копіюємо й нормалізуємо RGB...  
Підготовлено зображень: 14  
INSTANCE_DIR: /kaggle/working/instance_images
```

2. 1.3 Візуальна перевірка 4–8 навчальних зображень

Усі зображення містять один і той самий об'єкт, немає облич, приватних даних або чужого контенту, немає великої кількості майже однакових кадрів, об'єкт добре видно та фон не домінує над об'єктом. Отже, можемо починати training.



2.4 Вибір unique_token і class_noun

Налаштуємо унікальний токен та назву об'єкта

```
else:  
    UNIQUE_TOKEN = "jdsflightb" # змініть на власний рідкісний token  
    CLASS_NOUN = "bulb" | # змініть на клас вашого об'єкта
```

```
UNIQUE_TOKEN: jdsflightb  
CLASS_NOUN: bulb  
INSTANCE_PROMPT: a photo of jdsflightb bulb  
CLASS_PROMPT: a photo of a bulb  
VALIDATION_PROMPT: a photo of jdsflightb bulb in a studio
```

2.1.5 Baseline: генерація base model до навчання

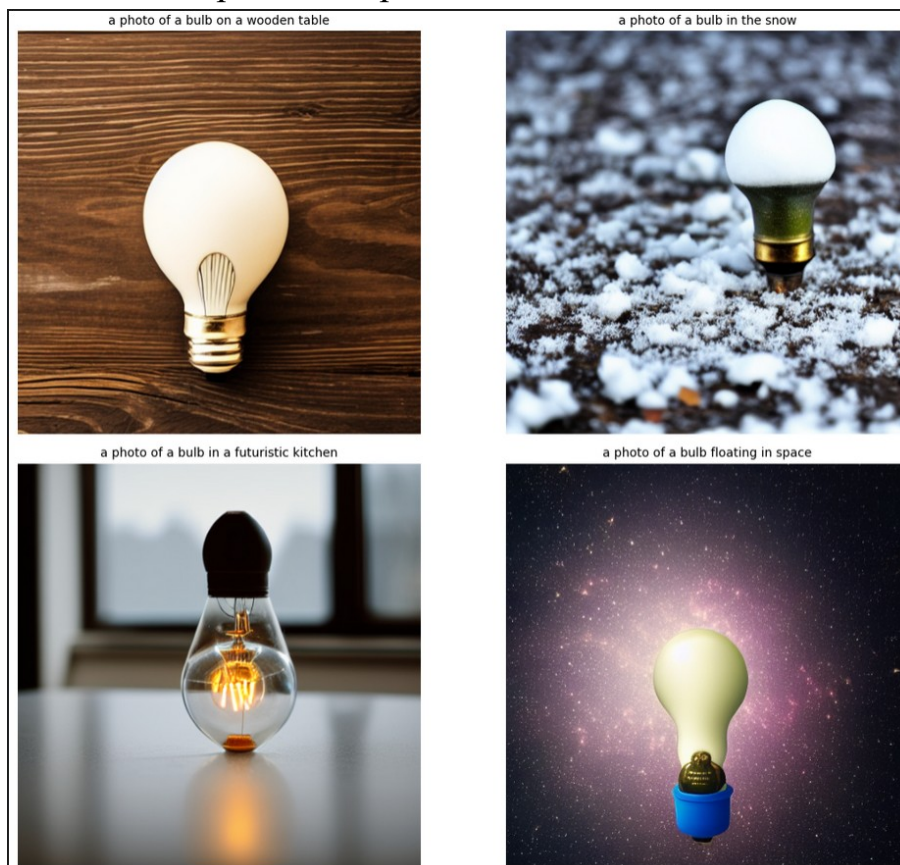
Перед fine-tuning згенеруємо зображення звичайною base model. Потім порівняємо їх із результатами LoRA. У baseline prompts не будемо використовувати unique_token, бо base model ще не знає об'єкта.

```
Baseline prompts:  
- a photo of a bulb on a wooden table  
- a photo of a bulb in the snow  
- a photo of a bulb in a futuristic kitchen  
- a photo of a bulb floating in space  
  
LoRA prompts:  
- a photo of jdsflightb bulb on a wooden table  
- a photo of jdsflightb bulb in the snow  
- a photo of jdsflightb bulb in a futuristic kitchen  
- a photo of jdsflightb bulb floating in space
```

Згенеруємо baseline-зображення.

```
100% ██████████ 35/35 [00:05<00:00, 7.55it/s]  
Saved: /kaggle/working/baseline_outputs/baseline_01.png | a photo of a bulb on a wooden table  
100% ██████████ 35/35 [00:04<00:00, 7.55it/s]  
Saved: /kaggle/working/baseline_outputs/baseline_02.png | a photo of a bulb in the snow  
100% ██████████ 35/35 [00:04<00:00, 7.48it/s]  
Saved: /kaggle/working/baseline_outputs/baseline_03.png | a photo of a bulb in a futuristic kitchen  
100% ██████████ 35/35 [00:04<00:00, 7.43it/s]  
Saved: /kaggle/working/baseline_outputs/baseline_04.png | a photo of a bulb floating in space
```

Переглянемо згенеровані зображення



2.1.6 Запуск DreamBooth LoRA training

Тепер запусимо офіційний script `/kaggle/working/train_dreambooth_lora.py`

Використаємо рекомендовані стартові значення

```
# ===== НАЛАШТУВАННЯ TRAINING =====  
  
RESOLUTION = 512  
TRAIN_BATCH_SIZE = 1  
GRADIENT_ACCUMULATION_STEPS = 1  
LEARNING_RATE = 1e-4  
MAX_TRAIN_STEPS = 500  
CHECKPOINTING_STEPS = 250  
VALIDATION_EPOCHS = 25  
NUM_VALIDATION_IMAGES = 2  
LORA_RANK = 4  
MIXED_PRECISION = "no"  
  
USE_8BIT_ADAM = False  
GRADIENT_CHECKPOINTING = True  
ENABLE_XFORMERS = True # xFormers не встановлено  
ALLOW_TF32 = True
```

Залишимо `MIXED_PRECISION = "no"` та `USE_8BIT_ADAM = False` задля уникнення подальших конфліктів. Та запусимо тренування моделі:

```
Steps: 100%|██████████| 500/500 [10:44<00:00, 1.29s/it, loss=0.201, lr=0.0001]  
Return code: 0  
Log saved to: /kaggle/working/training_live.log  
Training завершено успішно.
```

2.1.7 Inference після навчання

Тепер завантажмо base model і підключаємо LoRA weights із `LORA_OUTPUT_DIR`. Для fine-tuned prompts використаємо `unique_token`, наприклад: *a photo of `sks` puppy in the snow*

```
LoRA weights loaded from: /kaggle/working/dreambooth_lora_output  
100% ██████████ 35/35 [00:05<00:00, 6.42it/s]  
Saved: /kaggle/working/lora_outputs/lora_01.png | a photo of jdsflightb bulb on a wooden table  
100% ██████████ 35/35 [00:05<00:00, 6.34it/s]  
Saved: /kaggle/working/lora_outputs/lora_02.png | a photo of jdsflightb bulb in the snow  
100% ██████████ 35/35 [00:05<00:00, 6.22it/s]  
Saved: /kaggle/working/lora_outputs/lora_03.png | a photo of jdsflightb bulb in a futuristic kitchen  
100% ██████████ 35/35 [00:05<00:00, 6.16it/s]  
Saved: /kaggle/working/lora_outputs/lora_04.png | a photo of jdsflightb bulb floating in space
```

Переглянемо результати fine-tuned LoRA



2.1.8 Порівняння base model vs fine-tuned model

Порівняємо base model без знання нашого конкретного об'єкта та base model + LoRA weights із нашим unique_token. Оцінимо не тільки естетику, а й те, чи зберігся об'єкт і чи змінився контекст.





2.1.9 Аналіз результатів

prompt	recognizable	context	artifacts	overfitting	comments
a photo of jdsflightb bulb on a wooden table	5	5	немає суттєвих артефактів	слабкі	Об'єкт добре впізнається як лампочка, контекст дерев'яного столу виконано коректно
a photo of jdsflightb bulb in the snow	3	5	неприродність форми лампи	помірні	Модель успішно перенесла об'єкт у новий контекст, проте лампочка має криву форму вигляд
a photo of jdsflightb bulb in a futuristic kitchen	4	5	невеликі спотворення форми колби	помірні	Контекст кухні дотримано, але модель тяжіє до однакового дизайну лампочки
a photo of jdsflightb bulb floating in space	5	5	відсутні критичні артефакти	помірні	Об'єкт добре впізнається, модель успішно інтегрувала його у фантастичний сценарій

1. Чи впізнається навчальний об'єкт?

Навчальний об'єкт впізнається у всіх згенерованих зображеннях. Після застосування LoRA модель стабільно генерує лампочку зі схожими характерними ознаками незалежно від контексту сцени.

2. Чи змінився контекст відповідно до prompt?

Так, контекст змінюється відповідно до заданих prompt. Модель коректно розміщує об'єкт на дерев'яному столі, у снігу, на футуристичній кухні та в космосі, зберігаючи загальну ідентичність навчального об'єкта.

3. Який prompt спрацював найкраще?

Найкраще спрацював prompt: *a photo of jdsflightb bulb on a wooden table*

У цьому випадку об'єкт виглядає найбільш природно, добре вписується в сцену та має високу деталізацію без помітних артефактів.

4. Який prompt спрацював найгірше?

Найгірше спрацював prompt: *a photo of jdsflightb bulb in the snow*

Хоча контекст загалом виконано правильно, форма лампочки дещо відрізняється від інших результатів.

5. Чи видно ознаки overfitting?

Присутні незначні ознаки overfitting. У більшості результатів повторюються схожі характеристики лампочки: прозора скляна колба, яскрава нитка розжарювання та подібний стиль зображення. Проте модель все ще здатна адаптувати об'єкт до різних сцен, тому сильного перенавчання не спостерігається.

6. Чи видно ознаки underfitting?

Ознак underfitting не виявлено. Унікальний токен *jdsflightb* помітно впливає на генерацію, а навчальний об'єкт залишається впізнаваним у всіх протестованих сценаріях. Модель не генерує випадкову типову лампочку, а використовує ознаки об'єкта, отримані під час навчання.

2.1.10 Висновки: в ході роботи було успішно навчено LoRA-модель для генерації зображень із власним об'єктом — лампочкою. Модель коректно розпізнає об'єкт і переносить його в різні контексти відповідно до заданих prompt, хоча спостерігаються незначні ознаки перенавчання у вигляді повторення схожого стилю лампочки. Для подальшого покращення результатів можна використати більший і різноманітніший датасет, а також підібрати оптимальні значення кількості кроків навчання та швидкості навчання.

2.2 Відкриємо ноутбук stable_diffusion

2.2.1 Перевірка GPU та середовища

```
Python: 3.12.13 (main, Mar 4 2026, 09:23:07) [GCC 11.4.0]
Platform: Linux-6.6.122+-x86_64-with-glibc2.35
PyTorch: 2.10.0+cu128
CUDA available: True
GPU: Tesla T4
GPU memory: 14.56 GB
```

Аналогічним чином авторизуємося в Hugging Face та перевіримо успішність

```
HF_TOKEN отримано з Kaggle Secrets.
Hugging Face login виконано.
```

2.2.2 Demo image і mask для Img2Img/Inpainting

Оригінальний notebook Hugging Face використовує demo image і mask з GitHub. Якщо Kaggle Internet вимкнено або URL недоступний, створиться просте synthetic image.



2.2.3 Text-to-Image pipeline

Запустимо базову генерацію.

```
1 if RUN_TEXT2IMG:
2     prompt = "Palette knife painting of an autumn cityscape"
3     negative_prompt = "Oversaturated, blurry, low quality"
4
5     generator = torch.Generator(device=DEVICE).manual_seed(42)
6
7     output = pipe(
8         prompt=prompt,
9         negative_prompt=negative_prompt,
10        height=512,
11        width=512,
12        guidance_scale=8.0,
13        num_inference_steps=30,
14        generator=generator,
15    )
16
17    text2img_image = output.images[0]
18    show_images(text2img_image, titles=["Text-to-Image result"], col
19    save_images(text2img_image, "text2img_basic")
```

100% 30/30 [00:04<00:00, 7.56it/s]

Text-to-Image result



Saved:
/kaggle/working/stable_diffusion_lab_outputs/text2img_basic_00.png

Змінемо prompt та запустимо генерацію ще раз

```
1 if RUN_TEXT2IMG:
2     prompt = "A majestic dragon flying above a medieval village, detailed realistic fantasy art"
3     negative_prompt = "blurry, low quality, distorted anatomy"
```


Text-to-Image result



Змінимо `guidance_scale`.

Text-to-Image result



Базова генерація створила осінній міський пейзаж у стилі живопису мастихіном. Після зміни `prompt` модель згенерувала зовсім інший сюжет — середньовічне місто з драконом, що демонструє сильний вплив текстового опису на результат. Збільшення `guidance_scale` спотворило згенероване зображення.

2.2.4 Експеримент: вплив guidance_scale

Запустимо наступний cell і опишемо, який результат найкращий.



При `guidance_scale = 2.0` на голові собаки рожевий капелюх. Сама собака виглядає найбільш реалістично серед всіх трьох результатів, але схожа на опудало.

При `guidance_scale = 8.0` модель непогано дотримується заданого промту: рожевий капелюх чітко присутній на собаці, але при цьому пес виглядає не природньо і присутній зайвий другий капелюх.

При `guidance_scale = 12.0` модель занадто сильно орієнтується на текстовий опис. Рожевий капелюх стає дуже помітним, зображення виглядає ще менш природньо: окремі деталі собаки мають ознаки штучності, загальна реалістичність сцени знижується. Це демонструє, що надто високе значення `guidance_scale` може призводити до перенасичення впливу промту на результат генерації.

2.2.5 Компоненти Stable Diffusion pipeline

Виведемо список компонентів і знайдемо їх у змінній `pipe`.

```
Pipeline components:
vae                -> AutoencoderKL
text_encoder       -> CLIPTextModel
tokenizer          -> CLIPTokenizer
unet               -> UNet2DConditionModel
scheduler         -> PNDMScheduler
safety_checker     -> StableDiffusionSafetyChecker
feature_extractor  -> CLIPImageProcessor
image_encoder      -> NoneType
```

У пайплайні Stable Diffusion було знайдено такі основні компоненти: `vae` (`AutoencoderKL`), `text_encoder` (`CLIPTextModel`), `tokenizer` (`CLIPTokenizer`), `unet` (`UNet2DConditionModel`), `scheduler` (`PNDMScheduler`), `safety_checker` (`StableDiffusionSafetyChecker`) та `feature_extractor` (`CLIPImageProcessor`). Також присутній компонент `image_encoder`, але в даній конфігурації він має значення `None`, тобто не використовується.

Компонент VAE відповідає за кодування та декодування зображень між простором пікселів і латентним простором. Text Encoder (`CLIPTextModel`) перетворює текстовий `prompt` у числові представлення, які використовуються моделлю. Tokenizer розбиває текст на токени для подальшої обробки текстовим енкодером. UNet є основною нейронною мережею, яка виконує процес денойзингу та поступово генерує зображення з шуму. Scheduler керує послідовністю кроків генерації та визначає, як саме видаляється шум на кожній ітерації. Safety Checker перевіряє результати на наявність потенційно небажаного контенту. Feature Extractor готує зображення для роботи Safety Checker. Компонент `image_encoder` використовується в окремих типах пайплайнів, але в даній моделі він відсутній.

2.2.6 VAE: $\text{image} \rightarrow \text{latent} \rightarrow \text{image}$

1. Чому Stable Diffusion працює в latent space, а не напряму з pixel space?

Stable Diffusion працює в latent space, тому що це значно зменшує обсяг даних, які потрібно обробляти. Замість генерації зображення розміром $512 \times 512 \times 3$ безпосередньо в пікселях, модель працює зі стисненим представленням зображення, яке займає набагато менше пам'яті та потребує менше обчислювальних ресурсів. Це дозволяє прискорити навчання та генерацію зображень без суттєвої втрати якості.

2. Що означає shape [1, 4, 64, 64]?

Форма [1, 4, 64, 64] означає:

1 — розмір пакета (batch size), тобто одне зображення;

4 — кількість каналів у латентному представленні;

64×64 — просторовий розмір латентної карти.

Тобто зображення розміром 512×512 пікселів після проходження через VAE кодується в компактне представлення розміром $4 \times 64 \times 64$. Таким чином обсяг даних зменшується приблизно у 48 разів порівняно з початковим RGB-зображенням.

3. Навіщо потрібен `scaling_factor`?

`scaling_factor` використовується для правильного масштабування латентних представлень між VAE та дифузійною моделлю. Під час кодування латенти множаться на цей коефіцієнт, а перед декодуванням — діляться на нього. Завдяки цьому значення латентів залишаються в діапазоні, на якому була навчена модель Stable Diffusion, що забезпечує стабільну та якісну генерацію.

4. Що буде, якщо забути поділити `latents` на `scaling_factor` перед VAE decoder?

Якщо не поділити латенти на `scaling_factor` перед декодуванням, VAE отримає значення неправильного масштабу. У результаті декодоване зображення буде сильно спотвореним: можуть з'явитися артефакти, неправильні кольори, надмірний шум або повна втрата деталей. Тобто VAE не зможе коректно відновити зображення з латентного простору.

2.2.7 Tokenizer і Text Encoder

Запустимо наступну клітинку

```
Prompt: A painting of a flooble
Tokenizer max length: 77
input_ids shape: (1, 77)
First 20 decoded tokens:
['<|startoftext|>', 'a', 'painting', 'of', 'a', 'floo', 'ble', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>']
text_embeddings shape: (1, 77, 768)
```

Замінімо промт на кастомний і подивимося, як tokenizer розіб'є слова

```
prompt = "Dragon in the sky"

text_inputs = pipe.tokenizer(
    prompt
```

```
Prompt: Dragon in the sky
Tokenizer max length: 77
input_ids shape: (1, 77)
First 20 decoded tokens:
['<|startoftext|>', 'dragon', 'in', 'the', 'sky', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>', '<|endoftext|>']
text_embeddings shape: (1, 77, 768)
```

[+ Code](#) [+ Markdown](#)

Після токенизації prompt "Dragon in the sky" був розбитий на окремі токени: `<|startoftext|>`, `dragon`, `in`, `the`, `sky`, після чого були додані спеціальні токени `<|endoftext|>` для заповнення послідовності до максимальної довжини. Максимальна довжина послідовності для токенизатора CLIP становить 77 токенив, тому форма `input_ids` дорівнює `(1, 77)`, де 1 — кількість текстових запитів у пакеті.

Після проходження через текстовий енкодер було отримано тензор ознак розмірності `(1, 77, 768)`. Це означає, що для кожного з 77 токенив створюється векторне представлення розміром 768 елементів. Саме ці текстові ембедінги використовуються моделлю Stable Diffusion для керування процесом генерації зображення відповідно до змісту prompt.

2.2.8 UNet: прогнозування шуму

1. Чому форма `noise_pred` така сама, як форма `latents`?

Форма `noise_pred` збігається з формою `latents`, тому що завдання UNet полягає у прогнозуванні шуму для кожного елемента латентного представлення. Модель отримує зашумлені латенти на вході та повертає оцінку шуму тієї ж розмірності. Це дозволяє відняти передбачений шум від поточного латентного представлення і перейти до наступного кроку денойзингу.

2. Що таке `timestep`?

`timestep` — це номер поточного кроку процесу дифузії. Під час генерації модель поступово видаляє шум із латентного представлення за багато послідовних кроків. Значення `timestep` повідомляє UNet, скільки шуму наразі міститься в латентах і на якому етапі генерації вона знаходиться. Завдяки цьому модель може правильно прогнозувати шум для кожного конкретного кроку.

3. Чому UNet потрібні text_embeddings?

text_embeddings містять числове представлення текстового prompt, отримане за допомогою текстового енкодера CLIP. UNet використовує ці ембедінги через механізм cross-attention, щоб враховувати зміст текстового опису під час генерації зображення. Саме завдяки text_embeddings модель розуміє, що потрібно створити, наприклад, дракона в небі, а не будь-яке випадкове зображення. Без текстових ембедінгів генерація була б неконтрольованою і не залежала б від введеного prompt.

2.2.9 DIY Sampling Loop: ручна реалізація Text-to-Image

1. На якому етапі використовується prompt?

Prompt використовується на початку генерації, коли текст проходить через токенизатор і текстовий енкодер CLIP. У результаті отримуються text_embeddings, які передаються до UNet на кожному кроці дифузії. Саме вони визначають, які об'єкти, сцени та характеристики повинні з'явитися на зображенні.

2. Навіщо latents дублюються через torch.cat([latents] * 2)?

Латенти дублюються для одночасного виконання двох прогнозів UNet: без текстового опису (unconditional) і з текстовим описом (conditional). Це дозволяє отримати два набори передбаченого шуму за один прохід моделі та використати їх для механізму classifier-free guidance.

3. Що означають noise_pred_uncond і noise_pred_text?

noise_pred_uncond — це прогноз шуму, отриманий без використання текстового prompt. Він відображає те, що модель згенерувала б без додаткових інструкцій.

noise_pred_text — це прогноз шуму з урахуванням текстового опису. Він містить інформацію про те, як змінити зображення відповідно до заданого prompt.

4. Що робить формула classifier-free guidance?

Формула classifier-free guidance поєднує умовний і безумовний прогнози шуму:

$$\text{noise_pred} = \text{noise_pred_uncond} + \text{guidance_scale} * (\text{noise_pred_text} - \text{noise_pred_uncond})$$

Вона підсилює вплив текстового опису на генерацію. Чим більше значення `guidance_scale`, тим сильніше модель дотримується `prompt`. Це дозволяє отримувати зображення, які краще відповідають заданому опису.

5. Чому після `sampling` треба декодувати `latents` через VAE?

Після завершення `sampling` отримуються латентні представлення зображення, а не самі пікселі. Ці латенти знаходяться у стислому просторі, з яким працює Stable Diffusion. Щоб отримати звичайне зображення, яке можна переглядати або зберігати, необхідно передати латенти до декодера VAE, який перетворює їх назад у піксельний простір. Без цього кроку результат генерації залишиться лише у вигляді числових латентних даних.

2.2.10 Img2Img pipeline

Запустимо `Img2Img` із різними `strength` і порівняємо результати.

```
diy_img2img_image = diy_img2img(  
    pipe=pipe,  
    init_image=init_image,  
    prompt=diy_img2img_prompt,  
    negative_prompt=negative_prompt,  
    strength=0.25,  
    num_inference_steps=30,  
    guidance_scale=7.5,  
    seed=42,  
    height=512,  
    width=512,  
)
```

Input image



DIY Img2Img result



Замінемо strength на 0.6

```
)  
  
diy_img2img_image = diy_img2img(  
    pipe=pipe,  
    init_image=init_image,  
    prompt=diy_img2img_prompt,  
    negative_prompt=negative_prompt,  
    strength=0.6,  
    num_inference_steps=30,  
    guidance_scale=7.5,  
    seed=42,  
    height=512,  
    width=512,  
)
```

Input image



DIY Img2Img result



Замінемо strength на 0.9

Input image



DIY Img2Img result



Було проведено експеримент із різними значеннями параметра strength у режимі Img2Img. При початковому результаті модель майже повністю зберігала структуру вхідного зображення: собака залишалася на лавці, а зміни були мінімальними. Після збільшення strength до 0.6 модель почала суттєвіше змінювати зображення — замість собаки з'явилася людина на тій самій лавці, але загальна композиція та фон залишилися схожими на оригінал.

При strength = 0.9 вплив вихідного зображення став значно слабшим. Модель зберегла лише загальну ідею сцени з лавкою на відкритому просторі, але повністю змінила головний об'єкт і значно переробила оточення. Отримане зображення більше залежить від текстового опису, ніж від початкової фотографії.

Таким чином, зі збільшенням значення strength результат стає менш схожим на вхідне зображення і більше орієнтується на prompt. Низькі значення strength дозволяють зберігати більшість деталей оригіналу, тоді як високі значення дають моделі більше свободи для створення нового зображення

2.2.11 Контрольні питання до Img2Img

1. Чим Img2Img відрізняється від Text-to-Image?

У режимі Text-to-Image генерація починається з випадкового шуму, і модель створює зображення лише на основі текстового опису. У режимі Img2Img на вхід подається готове зображення, яке спочатку частково зашумлюється, а потім змінюється відповідно до prompt. Тому Img2Img дозволяє зберігати структуру або окремі деталі вихідного зображення.

2. Що робить параметр strength?

Параметр strength визначає, наскільки сильно модель буде змінювати початкове зображення. Низькі значення означають, що більша частина оригіналу буде збережена, а високі значення дозволяють моделі суттєво перебудувати зображення відповідно до текстового опису.

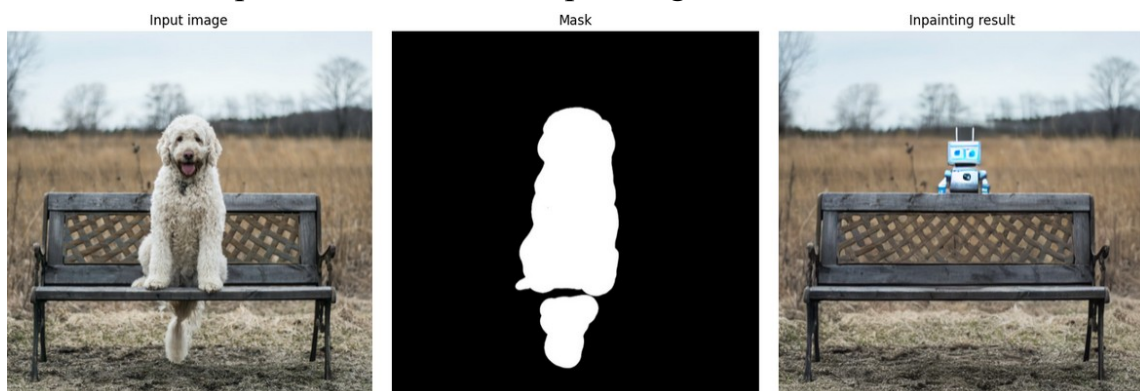
3. При якому strength input image зберігається найкраще?

Вхідне зображення найкраще зберігається при низьких значеннях strength. У проведеному експерименті найбільша схожість з оригіналом спостерігалася при найменшому значенні strength, де собака, лавка та загальна композиція практично не змінилися.

4. При якому strength prompt має найбільший вплив?

Найбільший вплив prompt має при високих значеннях strength. У проведеному експерименті при strength = 0.9 модель майже повністю змінила головний об'єкт і значно переробила сцену, тому результат більше відповідав текстовому опису, ніж вихідному зображенню.

2.2.12 Контрольні питання до Inpainting



1. Яка частина маски редагується: біла чи чорна?

У більшості реалізацій Stable Diffusion Inpainting редагується біла частина маски, а чорна частина залишається без змін. Біла область вказує моделі, які саме ділянки зображення потрібно згенерувати заново.

2. Чому inpainting потребує і image, і mask?

Image містить вихідне зображення, яке потрібно змінити, а mask визначає область для редагування. Без зображення модель не знала б контекст сцени, а без маски не розуміла б, які саме частини потрібно замінити або відновити.

3. Як зміниться результат, якщо маску зробити більшою?

Якщо збільшити область маски, модель буде змінювати більшу частину зображення. Це дає більше свободи для генерації нового контенту, але водночас підвищує ризик того, що результат буде менш схожим на оригінальне зображення або змінить важливі деталі сцени.

4. Що буде, якщо prompt не відповідає формі masked-області?

Якщо текстовий опис не відповідає формі або розміру замаскованої області, модель може створити неприродний результат. Наприклад, якщо в невелику область спробувати вставити великий об'єкт, окремі частини можуть бути обрізані, спотворені або виглядати непропорційно. У таких випадках часто з'являються артефакти та погіршується якість генерації.

2.2.13 Порівняння pipeline

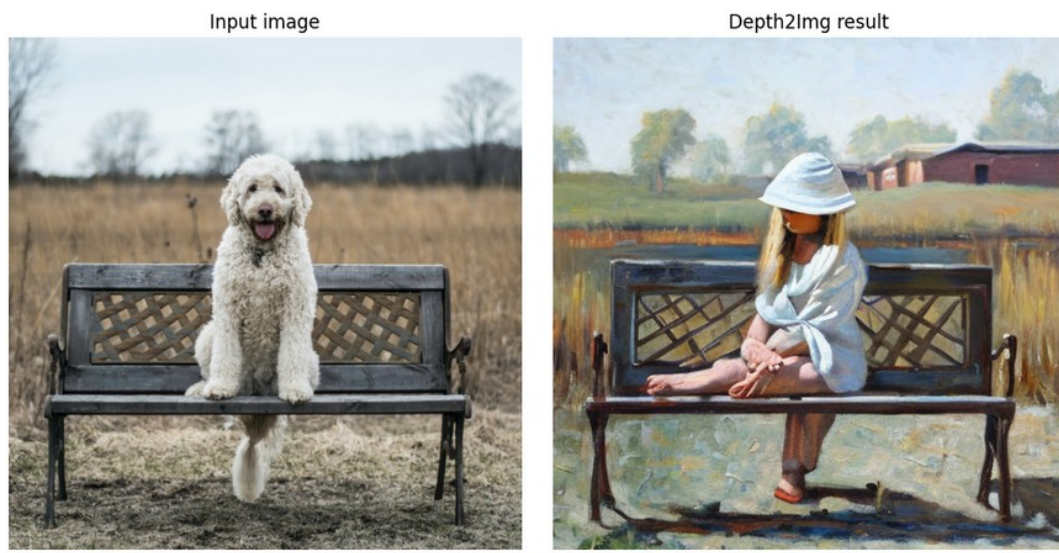
Input image



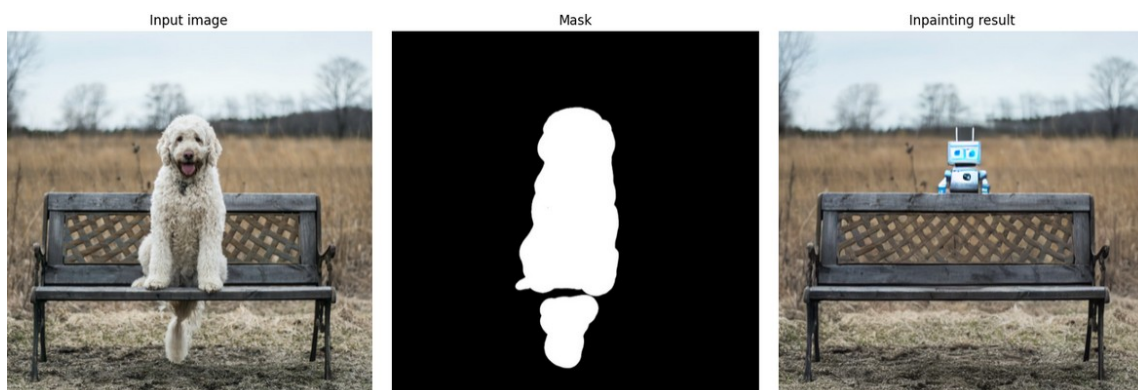
DIY Img2Img result



У випадку Img2Img збереглася загальна структура вихідного зображення: лавка, розташування об'єкта та фон залишилися дуже схожими на оригінал. Під впливом prompt собака була замінена на людину, однак композиція сцени майже не змінилася. Це демонструє, що Img2Img добре зберігає основні елементи вихідного зображення, одночасно дозволяючи модифікувати його відповідно до текстового опису.



У результаті Depth2Img також збереглася просторова структура сцени, оскільки модель використовує карту глибини. Лавка залишилася на тому самому місці, а розташування об'єкта в просторі відповідає оригінальному зображенню. Водночас собака була замінена на людину, а стиль зображення змінився на художній живописний. Завдяки використанню інформації про глибину модель добре зберегла перспективу та взаємне розташування об'єктів.



У випадку Inpainting більша частина початкового зображення залишилася незмінною: фон, лавка та навколишнє середовище були повністю збережені.

Зміни відбулися лише в області, визначеній маскою. Собака була видалена, а на її місці згенеровано невеликого робота відповідно до prompt. Це дозволило локально відредагувати лише потрібну частину зображення без впливу на решту сцени.

Серед розглянутих підходів Inpainting найкраще підходить для редагування окремої частини зображення, оскільки він дає можливість точно вказати область змін за допомогою маски та зберегти всі інші елементи зображення без модифікацій.

3. Висновки: в ході виконання лабораторної роботи було досліджено принципи роботи моделі Stable Diffusion та виконано генерацію й редагування зображень різними методами. Отримані результати підтвердили вплив параметрів генерації на кінцевий результат.

4. Посилання

Посилання на файли в репозиторії гіт

https://github.com/NureTymoshchenkoVladyslava/GNMS_Labs/tree/main/Lb3